🚆 PAMOJA RAIL SERVICES

Commercial Railway Operations, Ticket Reservation & M-Pesa STK Push

System

Project Name: Final Year Railway Management System

Technology: Pure Python 3 (Full-Stack Desktop Architecture)

Role-Based Access: Administrator & Passenger Portals

Payment Integration: Safaricom M-Pesa STK Push Simulation

1. Executive Summary & Architecture Overview

Pamoja Rail Services is a full-featured commercial railway management and passenger ticket reservation desktop system. It was engineered in **Python 3** adopting a unified **Full-Stack Architecture** where Python concurrently powers the **Presentation (Frontend UI) Layer** and the **Business Logic & Data Persistence (Backend) Layer** without requiring third-party frameworks,

web servers, or external database engines.

System Layer	Technologies Used	Primary Responsibilities
Frontend Layer Presentation	Python `tkinter` & `ttk` (Themed Toolkit)	Interactive graphical windows, card layouts, real-time KPI metrics, form validation, styled data tables (`Treeview`), and pop-up modal dialogs.
Backend Layer Logic & State	Python `DataStore` Service, OOP Controller	Role-Based Access Control (RBAC), fare calculation engine, ticket reference generation, manifest aggregation, user lifecycle management.
Data Layer Persistence	JSON Database Engine (`pamoja_data.json`)	ACID-like localized state persistence across app reboots for users, train fleet schedules, and passenger ticket reservations.
Fintech Gateway Payment	Safaricom M-Pesa Daraja STK Push Simulation	Handset PIN push prompt simulation, Paybill `888822` verification, validation delay, and authentic receipt code generation (`QHK...`).

2. How Python Acts as Both Frontend & Backend

A. Python as the Frontend (Presentation Layer)

In conventional web applications, the frontend requires HTML, CSS, JavaScript, and a web browser. In this project, **Python serves directly as the frontend GUI engine** using the native `tkinter` and `ttk` standard modules:

- **Window Lifecycle & Event Loop (`tk.Tk` & `mainloop()`):** Initializes an OS-level graphical window (X11 on Linux, Win32 on Windows, Cocoa on macOS) and listens asynchronously for user clicks, keyboard submissions, and tab changes.
- **Component-Based Views:** Modular view methods (e.g. `show_login_screen()`, `nav_dashboard()`, `nav_bookings()`, `nav_trains()`) dynamically destroy and reconstruct container frames, providing a Single-Page Application (SPA) experience inside a native desktop frame.
- **Advanced Data Tables (`ttk.Treeview`):** Renders high-performance tabular datasets with multi-column sorting, row selection hooks, custom padding, and clean scrollable rows.
- **Modal Dialogs & Simulated Hardware (`tk.Toplevel`):** Spawns focused child dialogs with

`transient()` and `grab_set()` to lock parent inputs while taking passenger booking inputs or simulating an M-Pesa smartphone push notification.

- **Design System:** Structured corporate branding matching Kenyan rail standards—Crimson Red (`#B91C1C`), Imperial Gold (`#D97706`), Porcelain White (`#FFFFFF`), and Safaricom Green (`#16A34A`).

B. Python as the Backend (Business Logic & Database Service)

The backend functions through the encapsulated `DataStore` class, managing all business rules, mathematical computations, and persistence:

- **Role-Based Access Control (RBAC):** Validates hashed credentials against authorized security roles:
  - **Administrator:** Unrestricted access to financial analytics, fleet schedule CRUD, passenger manifests, and member suspension/deletion.
  - **Passenger Member:** Self-service ticket booking, personal journey history review, and live timetable lookup. Restricted from administrative functions.
- **Dynamic Fare Computation Engine:** Computes trip charges dynamically based on distance routes and chosen seating tier:

```
# Automated Tariff Calculation Logic if "Economy" in class_selection: fare = 1500
# Standard Economy Tariff (KES) elif "First Class" in class_selection: fare =
3000 # First Class Recliner Tariff (KES) else: fare = 6000 # VIP Sleeper Suite
Tariff (KES)
```

- **Persistent JSON Database Engine:** Whenever bookings are confirmed or trains modified, the backend serializes the in-memory object graph directly to `pamoja_data.json` using Python's standard `json` library with atomic write safety.
- **Analytical Aggregation:** Performs real-time summaries over booking arrays to derive customer metrics: total journeys taken, cumulative revenue collected, and modal seating preference.

3. Safaricom M-Pesa STK Push Payment Gateway

> **Fintech Architecture Highlight: Lipa Na M-Pesa Online (Daraja API STK Push)**

> Commercial transport systems in Kenya (e.g., SGR Madaraka Express) rely on M-Pesa as

> their primary payment mechanism. Our system incorporates an end-to-end **simulated**

> **STK Push payment lifecycle**.

Step-by-Step STK Push Payment Workflow:

1. **Initiation:** Passenger selects Route (e.g., *Nairobi → Mombasa*), Travel Class (e.g., *First Class*), and Date, then clicks **"📲 Pay via M-Pesa STK Push"**.
2. **Gateway Modal (`show_mpesa_stk_dialog`):**
    - Merchant: **Pamoja Rail Services Ltd**
    - Business Paybill: **888822**
    - Generated Ticket Ref: **PRS-940X**
    - Prompt displays total amount (e.g. `KES 3,000.00`).
    - Passenger inputs Safaricom Mobile Number (e.g. `0712345678`).
3. **STK Push Handset Simulation (`show_simulated_phone_prompt`):**
    - A smartphone-style prompt pops up simulating the SIM Toolkit / push notification: *"Do you want to pay KES 3,000 to PAMOJA RAIL SERVICES for Account PRS-9406?"*
    - User inputs their 4-digit M-Pesa PIN (masked entry field).
4. **Daraja API Verification Simulation:**
    - System displays *"⏳ Validating PIN with Safaricom Daraja API..."* with a controlled asynchronous delay.
    - Generates a cryptographically random Safaricom Alpha-Numeric Receipt Code (e.g., `QHK9412M7B`).
5. **Confirmation & Ticket Issuance:**
    - Transaction status is updated to `Paid (M-Pesa: QHK9412M7B)`.
    - Passenger receives instant confirmation pop-up containing the receipt code, phone number, and ticket reference.
    - Booking is persisted to `pamoja_data.json` and immediately reflects in the Administrator Ledger and Dashboard revenue tally.

```
# Algorithm for M-Pesa Receipt Generation (Authentic 10-character code) chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789" receipt = "QHK" + "".join(random.choice(chars) for _ in range(7)) # Output Example: QHK8291X42
```

4. Presentation Script & Demonstration Walkthrough for Lecturer

Use the following structured outline when presenting your project to your lecturer:

Phase 1: System Purpose & Login Demonstration (2 Minutes)

- *"Good morning/afternoon. Today I present **Pamoja Rail Services**, an automated railway booking and fleet operations management system."*
- **Show the Login Screen:**
  - Point out the clean red-and-white branding.
  - Demonstrate the role toggle: **Administrator** vs. **Member Portal**.
  - Explain that security is enforced through Role-Based Access Control (RBAC).
  - Sign in using the pre-seeded admin credentials: `admin` / `admin123`.

Phase 2: Administrator Dashboard & Operations (3 Minutes)

- **Live KPI Metrics:** Explain how the dashboard automatically counts Total Bookings, Active Trains, Registered Members, and Delayed/Alert statuses in real time.
- **Fleet Management (`Train Fleet & Routes`):**
  - Show the train table: Express 101, Inter-County 204, Coastliner 305, etc.
  - Click **"🔄 Toggle Status"** to change a train from *On Time* to *Delayed* or *Maintenance*. Show how the dashboard KPI updates immediately.
  - Click **"➕ Add New Train Schedule"** and create a new route.
- **User Oversight (`User Management`):**
  - Show registered members. Explain security controls: Admin can **Suspend / Reinstate** accounts or permanently remove malicious accounts. Point out that the Primary Admin account is protected from accidental deletion.

Phase 3: Passenger Ticket Booking & M-Pesa STK Push Simulation (3 Minutes)

- *"Now let us examine the passenger reservation and payment experience."*
- Click **"Sign Out"** and log in as passenger member `john.kamau` / `member123` (or click **Register Member Account**).
- Navigate to **"🎟️ Book a Ticket"**:
  - Select Route: *Nairobi - Mombasa*.
  - Select Seating Tier: *First Class (KES 3,000)*.
  - Click **"📲 Pay via M-Pesa STK Push"**.

- **The Live Payment Demo:**
    - Show the Lipa Na M-Pesa Online modal with Paybill `888822` and generated Account Ref `PRS-940X`.
    - Click **"Send STK Push Prompt"**.
    - Show the simulated phone prompt popping up. Enter PIN `1234`.
    - Click **"Authorize & Pay"**. Point out the API simulation and the resulting authentic M-Pesa receipt code (e.g. `QHK...`).
    - Show that the booking appears in **"My Bookings"** marked as `Paid (M-Pesa)`!

Phase 4: Technical Architecture & Questions (2 Minutes)

- *"Technically, this entire system is built in **pure Python**—handling both GUI presentation via Tkinter and data operations via our custom DataStore service."*
- Explain the **zero-dependency portability**: *"It requires no third-party pip packages, runs seamlessly on any OS, and stores data safely in structured JSON."*

5. Summary of System Credentials & Source Files

Account Role	Username	Password	Permissions
Administrator	`admin`	`admin123`	Full operations dashboard, fleet management, all passenger reservations, financial ledger, user suspension/deletion.
Member Passenger	`john.kamau`	`member123`	Self-service ticket booking, M-Pesa STK push payment, personal booking history, train timetable lookup.
Member Passenger	`faith.mwangi`	`member123`	Personal reservation management and M-Pesa booking.
New Passenger	*Self-Register*	*User Choice*	Created instantly via the "Register Member Account" button.

Project File Map:

- **`main.py`:** Core application containing all frontend UI views (Tkinter/ttk), the MVC controller, business rules, and the Safaricom M-Pesa STK push simulation.

- **`pamoja_data.json`:** Persistent data file storing user records, train fleet schedules, and passenger bookings.
- **`run.bat`:** 1-click Windows launcher that auto-detects Python and launches `main.py`.
- **`README.md`:** Technical setup instructions and command reference.
- **`.vscode/`:** VS Code configuration for one-key debugging and task execution (`F5` / `Ctrl+Shift+B`).
